School of Computing
Queen's University

CISC235

Winter 2026

Assignment 4

Deadline: Mar 25, 2026, 11:59 PM

Graphs are frequently used to model communications networks, and in this context the length of the shortest paths between vertices is of great importance.

In this assignment we will be assuming that the time to transmit a message from vertex x to vertex y is determined completely by the number of edges in the shortest path between x and y.

Given a connected graph G, it is natural to ask what the maximum vertex-to-vertex message time is. In other words ...

Assuming that all messages are sent by the shortest possible paths, what is the maximum time for a message to get from one vertex to another?

To formalize this question, we can define some terms:

Let **distance(**x,y**)** = the number of edges in a shortest path between vertex x and vertex y

distance(x,y**)** = infinity if there is no path between x and y (this will never happen if G is connected)

Distance (x, x) = 0

Let diameter(G) = max(distance(x,y) | all vertices x and y in V)

Our question is now simpler to state: Given a connected graph G, what is diameter(G)?

This is relatively easy to compute. Recall that the Breadth-First-Search (BFS) algorithm, when started at a vertex v, reaches each vertex y using a minimum number of edges – i.e. BFS(v) finds the shortest paths from v to all other vertices. To find the diameter of G we run BFS on each vertex in turn, and keep track of the maximum distance of any other vertex from the start vertex each time.

Complete the following sub-tasks. Please check the appendix for how to create such

graphs.

Part 1:
Implement the algorithm described above for generating random connected graphs.

To demonstrate correctness, generate a random connected tree on 8 vertices, print the adjacency lists and print the diameter of the tree, then add (about) 8 more edges, print the new adjacency lists for the graph, and print the diameter of the graph.

Part 2: Conduct the following experiment.
choose a sample size k – it should probably be 10 or more, but you may find that for the
larger graphs the computation of the diameter is quite time-consuming – if so, using a
smaller sample is acceptable
For n = 100, 200, 400, 800, 1600
    for test = 1 to k:
        generate a tree on *n* vertices
        compute and record the diameter of the tree
        add *n* randomly selected edges (either approximately or exactly)
        compute and record the diameter of the graph
    compute the average tree diameter and average graph diameter for your graphs with *n* vertices

Create a table or chart relating *n* to the average tree diameter and average graph diameter. Can you discern a relationship? How do the average diameters appear to grow as a function of *n*?

Part 3: Kruskal's vs. Breadth-First-Search (BFS).
Implement the Kruskal's Minimum Spanning Tree algorithm
Compare the efficiency of the Kruskal's algorithm and BFS by performing the below experiments:

For n = 20, 40, 60:
    for test = 1 to k: #k is an input parameter you could decide yourself, it should be probably 10 or more.
        generate a tree on *n* vertices
        generate a random graph with *n* vertices
        use BFS to find a spanning tree (let its total weight be B)
        use Kruskal to find a spanning tree (let its total weight be P)
        compute Diff #Diff is the percentage by which B is larger than P

    compute the average of the values of Diff for this value of *n*

For instance, if B=20, P=10, Diff = (20-10)/10 = 100%

Report the average value of Diff for each value of *n*. What could you conclude from the observed results by varying the *n* values in this experiment?

Appendix – Generating Random Connected Graphs

There are different ways to do this – here is a very simple method to generate a connected graph on n vertices:

1. Generate a random tree on n vertices.
2. Add randomly selected edges.

Of course, that just raises another problem: how to generate a random tree on n vertices.

Fortunately, that's very easy:

```
1.  Let the vertices be numbered 0,1, ..., n-1
2.  for i = 1 to n-1
        Randomly choose p in the range [0,i-1]
        make i and p neighbors
```

Adding randomly selected edges is also very easy. Suppose we want to add up to k edges:

```
for i = 1 to k
    Randomly choose v1 in the range [0,n-1]
    Randomly choose v2 in the range [0,n-1]
    if v1 != v2, and v1 and v2 are not already neighbors
        make v1 and v2 neighbors
```

Note that this will not necessarily add exactly k edges. If you want to add exactly k edges, you can replace the for loop with a while loop that keeps going until k edges have been added.